Introduction to Large Language Models

Marco Calamo

Laboratory of Advanced Programming

SAPIENZA
UNIVERSITÀ DI ROMA

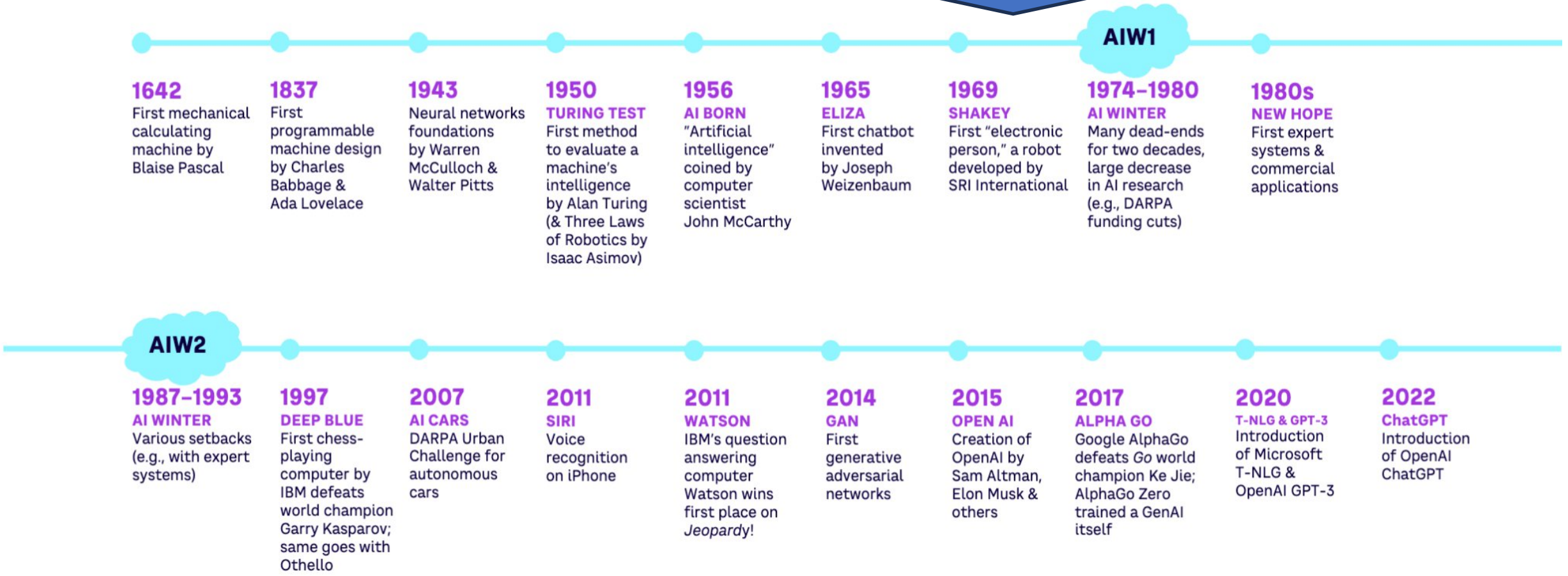
Introducing AI, GenAI, LLMs

A quick&dirty introduction with the eyes of an IS engineer

SAPIENZA
UNIVERSITÀ DI ROMA

AI Timeline

Artificial intelligence (AI) is technology that enables computers and machines to simulate human intelligence and problem-solving capabilities
Source: *https://www.ibm.com/topics/artificial-intelligence*



Source: https://www.adlittle.com/en/insights/report/generative-artificial-intelligence-toward-new-civilization

SAPIENZA UNIVERSITÀ DI ROMA

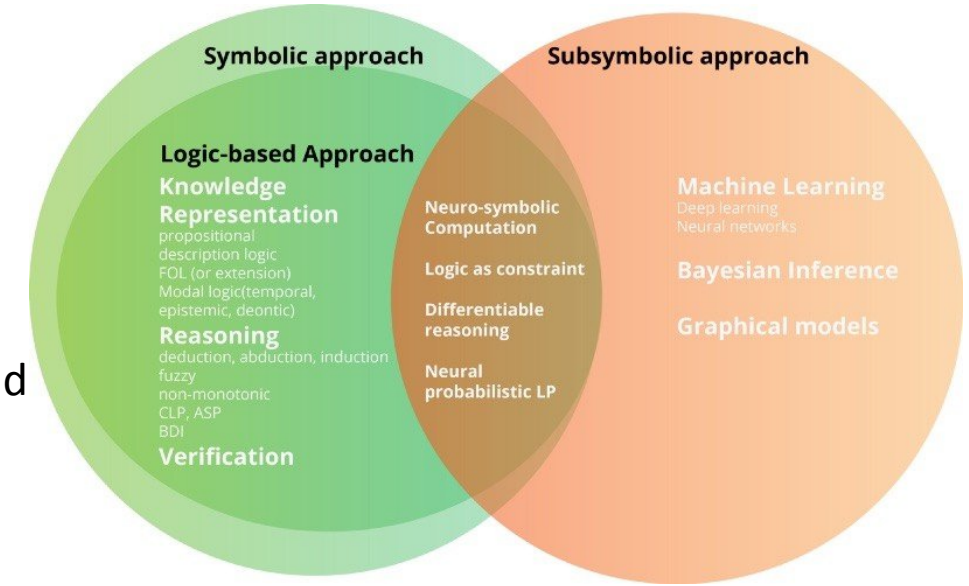
Symbolic and Subsymbolic AI

Symbolic AI

also called rule-based or traditional AI, it refers to an artificial intelligence approach that relies on a base of **knowledge represented in the form of symbols or rules**

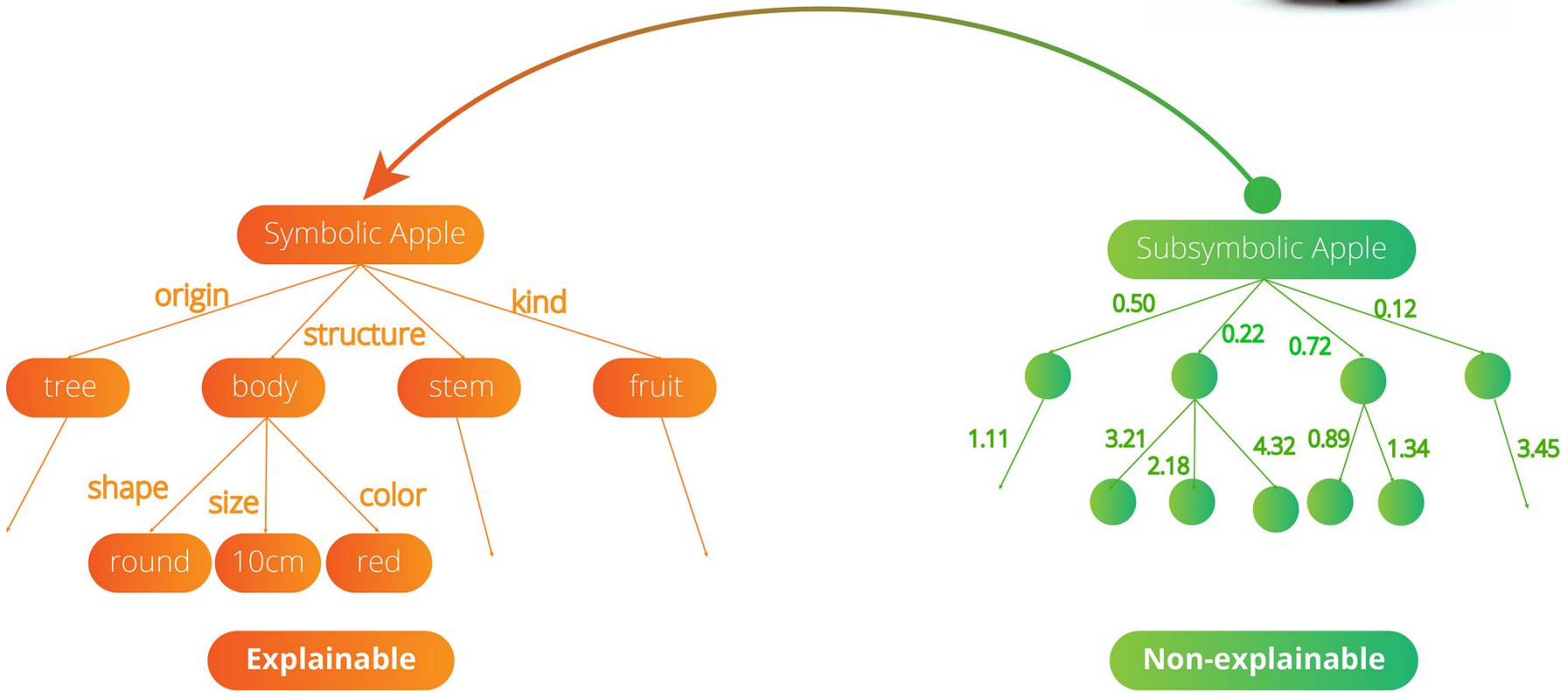
Subsymbolic AI

it refers to the approach that focuses on creating intelligent behavior by simulating the behavior in biological systems. It deals with the processing and analysis of data using **numerical methods and statistical models** to solve problems without the need for explicit rules or algorithms



Source: https://www.vizio.ai/blog/black-box-and-white-box-models-towards-explainable-ai

Symbolic and Subsymbolic AI



Source: https://towardsdatascience.com/symbolic-vs-subsymbolic-ai-paradigms-for-ai-explainability-6e3982c6948a

History of Language Models

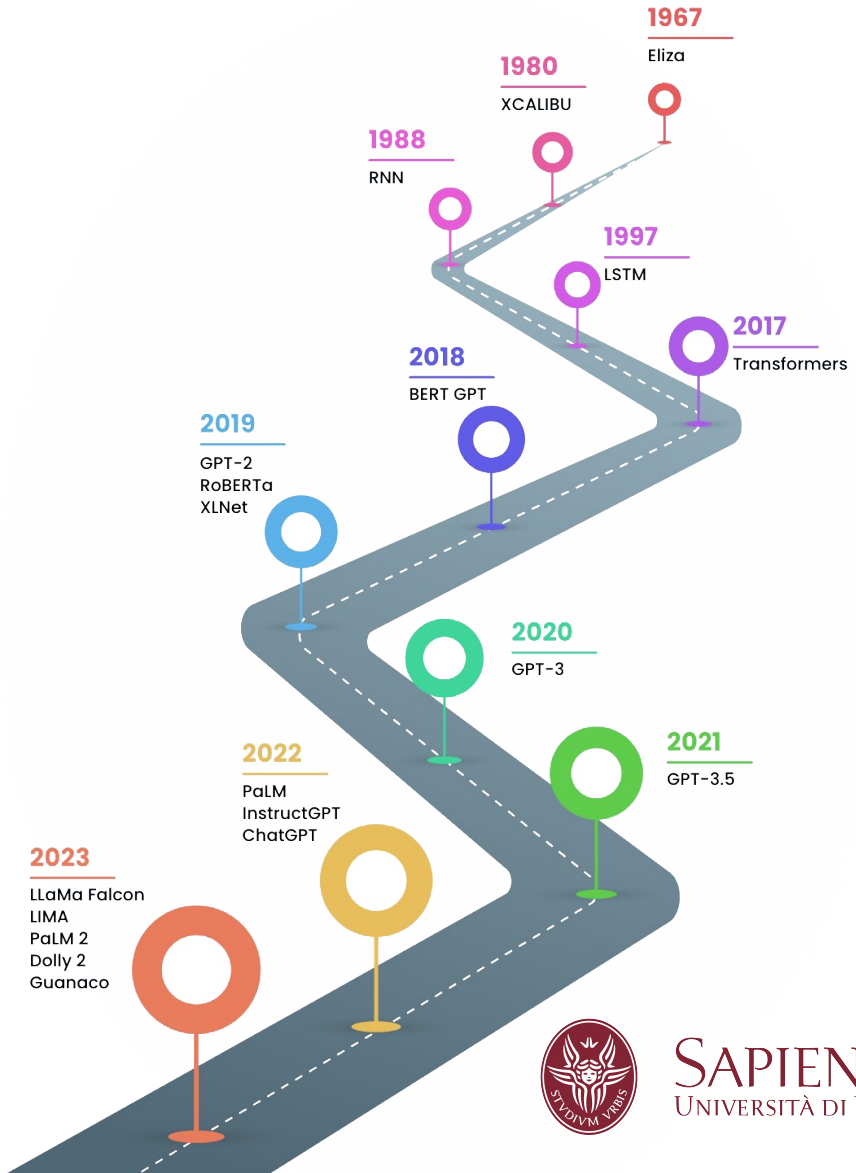
Source: *Evolution of Language Models: From Rules-Based Models to LLMs*

What is a language model?

A language model is a probabilistic model of a natural language[1]. Since the 60s researchers in the field of Artificial Intelligence have tried to *teach* the natural language to the machine.

A few milestones:

- **Eliza:** is the world's first chatbot. It simulated conversation by using a pattern matching and substitution methodology that gave users an illusion of understanding on the part of the program (*Symbolic AI*)
- **RNN:** is a type of neural network characterized by direction of the flow of information between its layers (*Subsymbolic AI*)
- **LSTM:** is a type of RNN aimed at dealing with the vanishing gradient problem present in traditional RNNs[2] (*Subsymbolic AI*)
- **Transformers:** the major breakthrough in the field that made possible *large* language models (*Subsymbolic AI*)



[1] *Jurafsky, Dan; Martin, James H. "N-gram Language Models". (2024)*
[2] *Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." Neural computation 9.8 (1997): 1735-1780.*

SAPIENZA UNIVERSITÀ DI ROMA

Large Language Models - Basics

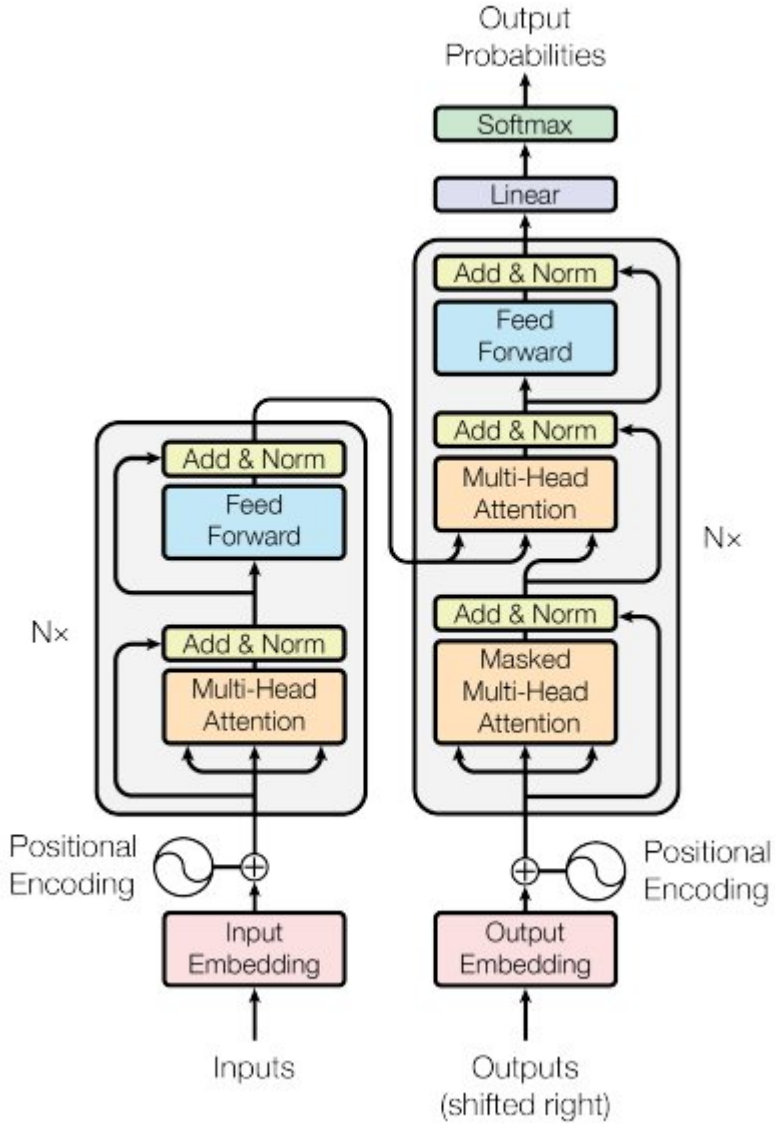
Transformers

A **transformer** is a deep learning architecture developed by Google and based on the **multi-head self attention mechanism**[1] . They have the advantage of having no recurrent units. It is originally composed by an **encoder** and a **decoder** part.

What makes a model "large"

The introduction of the transformer architecture made possible the training of models with **billions of parameters**. Bigger models as of today are reaching one trillion parameters.

Even though the transformer architecture has been improved over the years (mostly because of $O(n^2)$ computation complexity), **it is still the base of modern Large Language Models**.

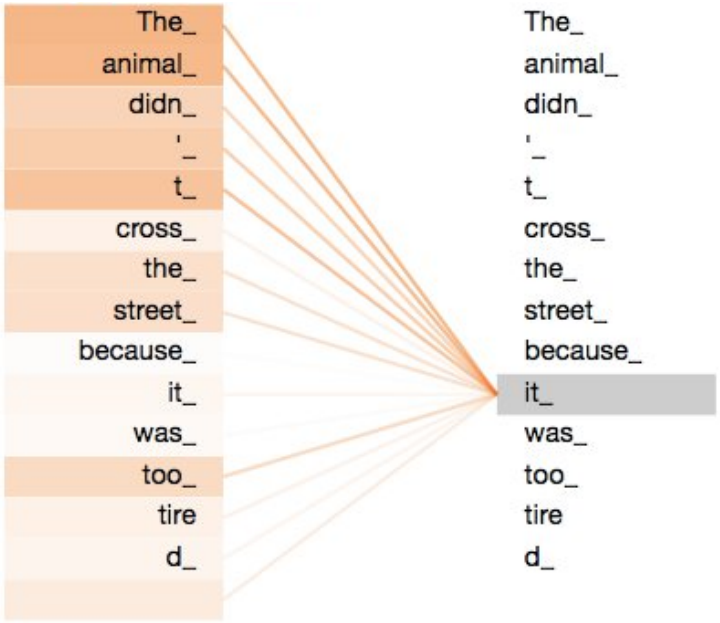


Encoder and Decoder Self-Attention[1]

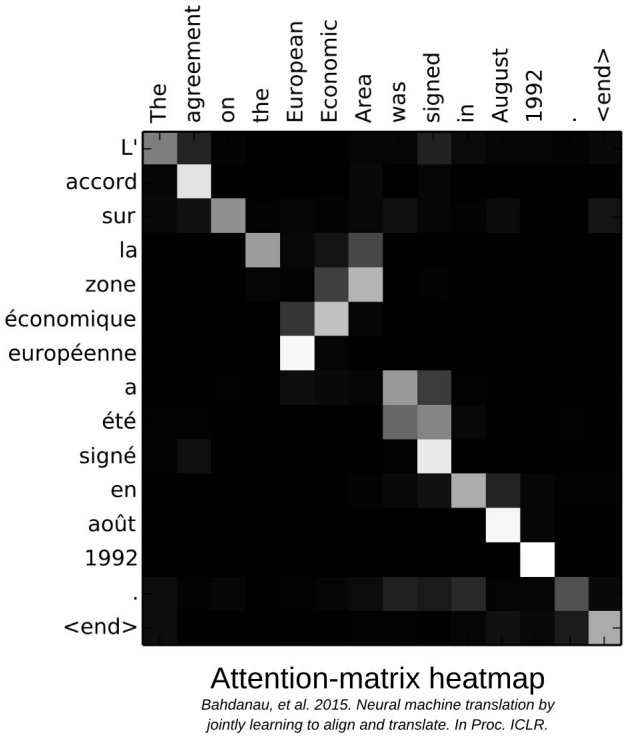
[1] *Vaswani, Ashish, et al. "Attention is all you need." (2017)*

SAPIENZA UNIVERSITÀ DI ROMA

Visualizing Attention



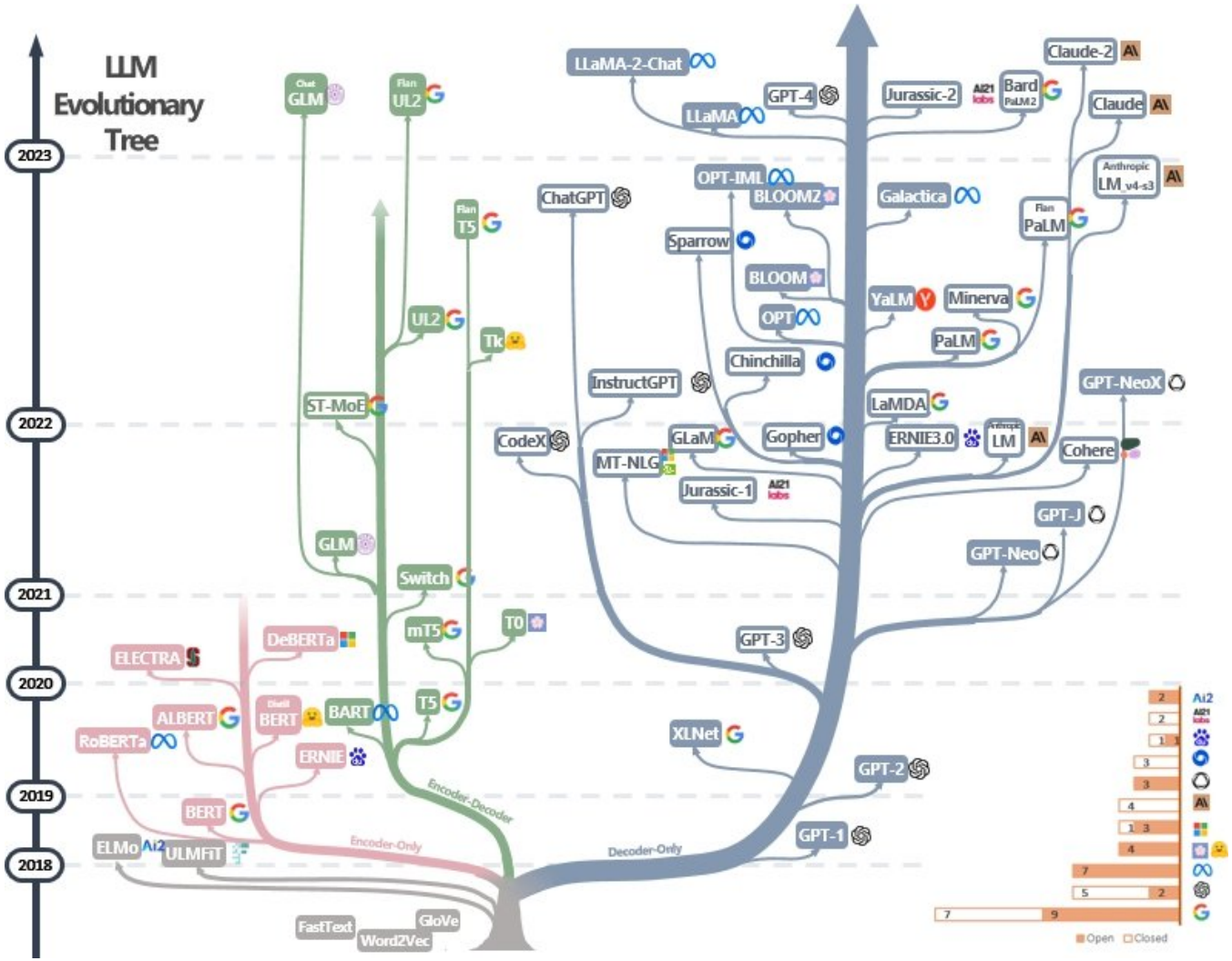
Self-attention



Cross-attention

Source: https://jalammar.github.io/illustrated-transformer/

LLM - Architecture and Evolution



LLM Evolutionary Tree[1]

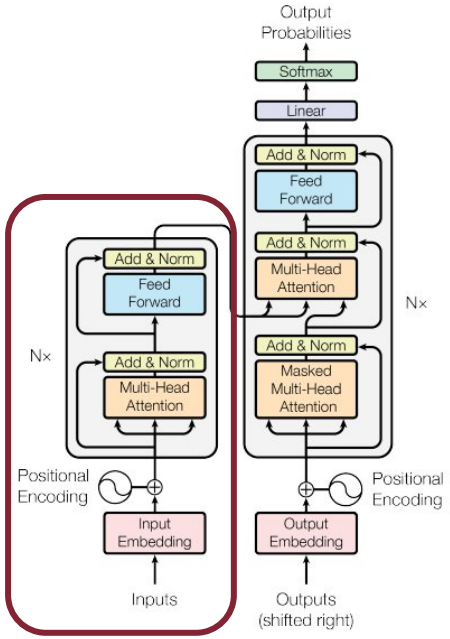
[1] *Yang, Jingfeng, et al. "Harnessing the power of llms in practice: A survey on chatgpt and beyond (2024)*

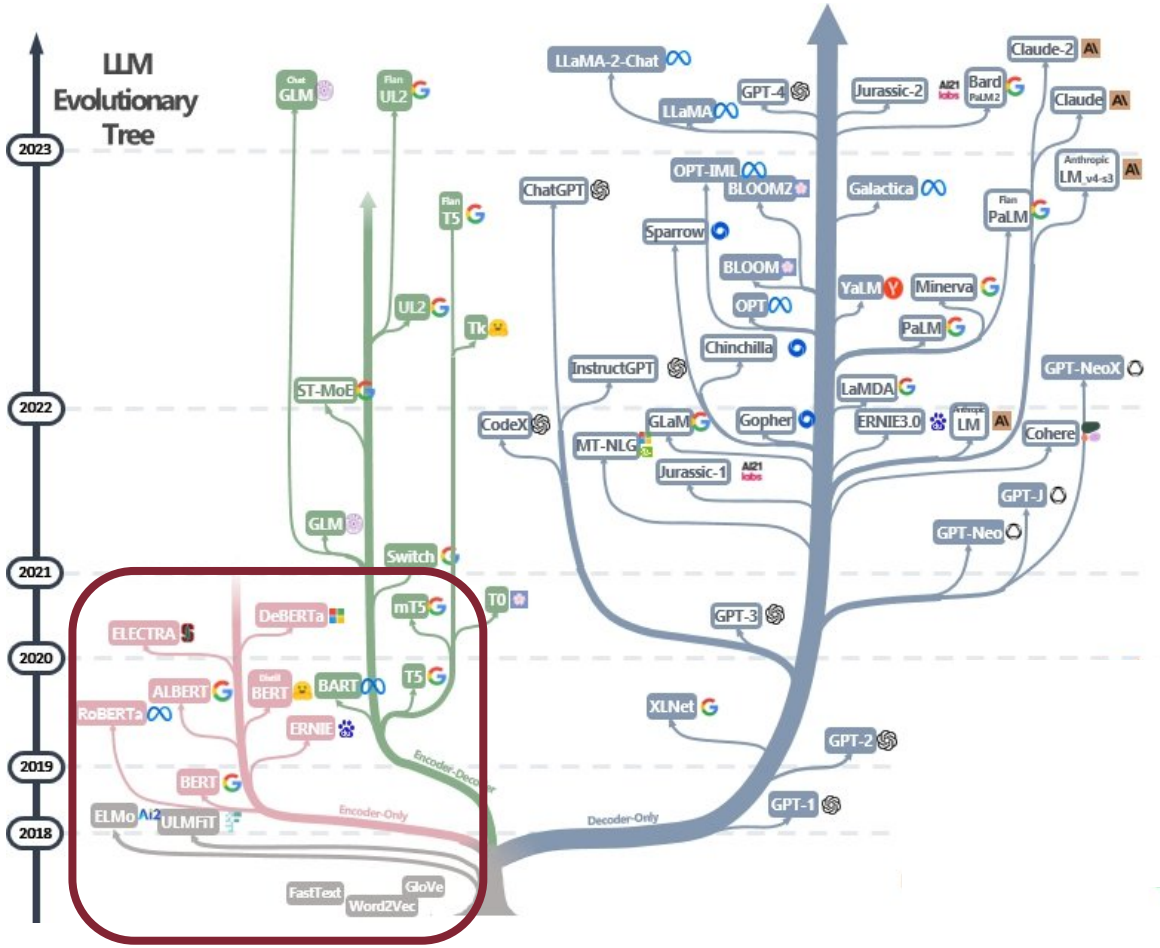
SAPIENZA UNIVERSITÀ DI ROMA

LLM - Architecture and Evolution

Encoder-Only Architecture

The transformer encoder architecture takes in a sequence of tokens and produces a fixed-size vector representation of the entire sequence, which can then be used for **classification**

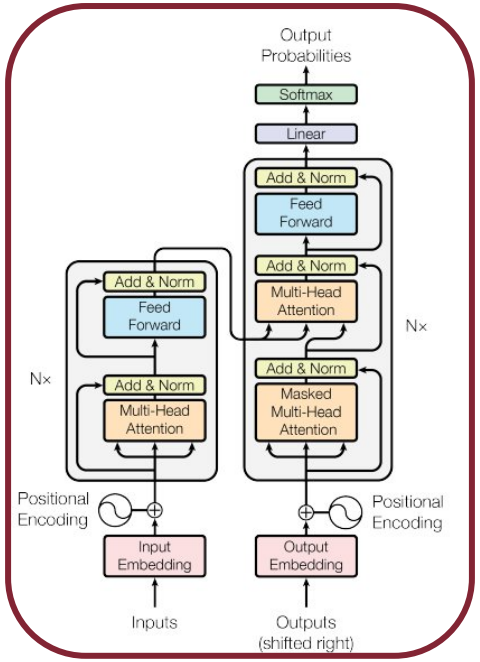


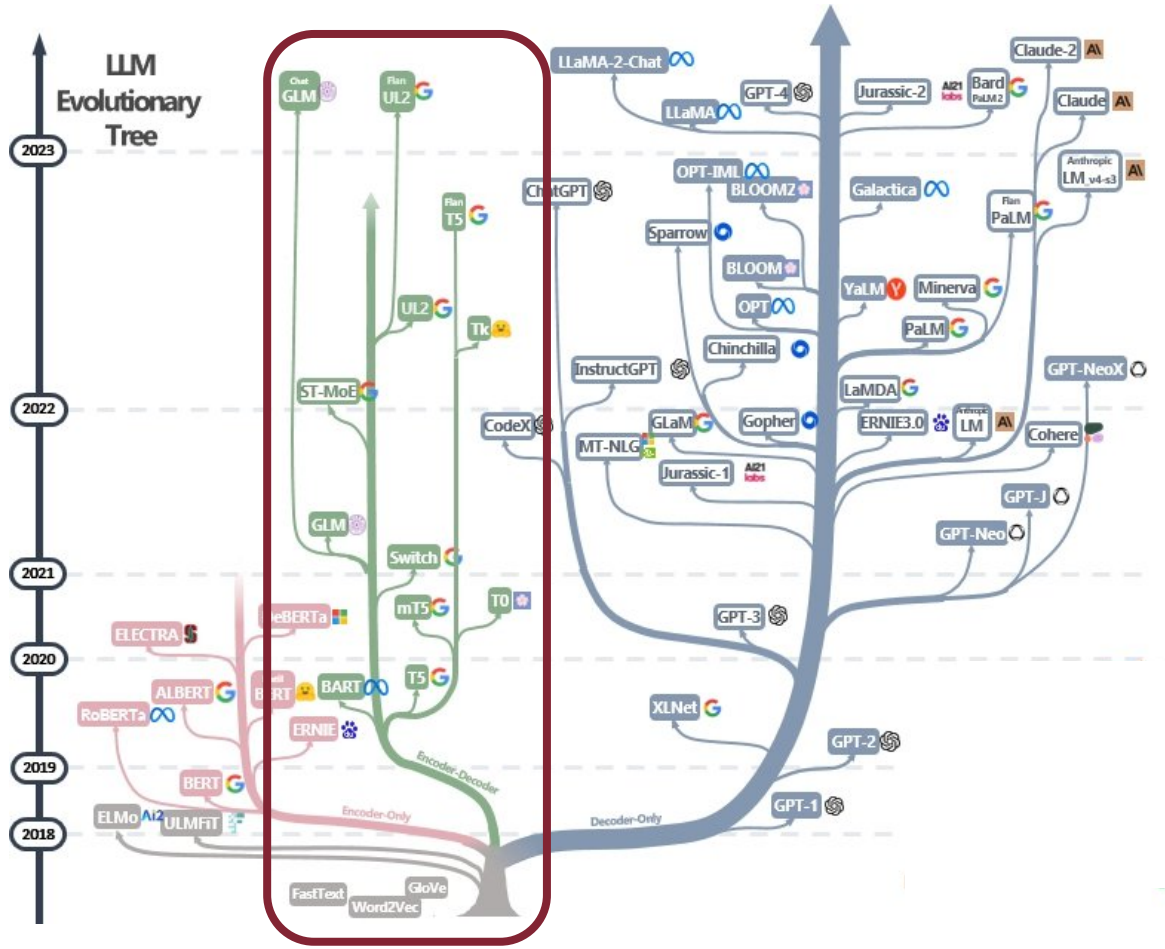


LLM - Architecture and Evolution

Encoder-Decoder Architecture

The transformer encoder-decoder architecture is used for tasks like **language translation**, where the model must take in a sentence in one language and output a sentence in another language

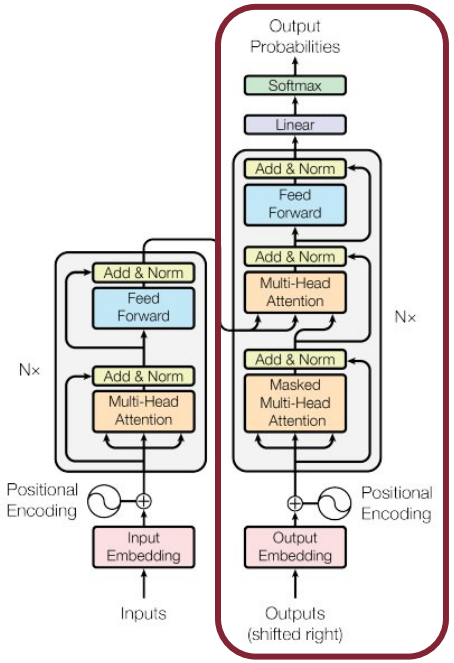


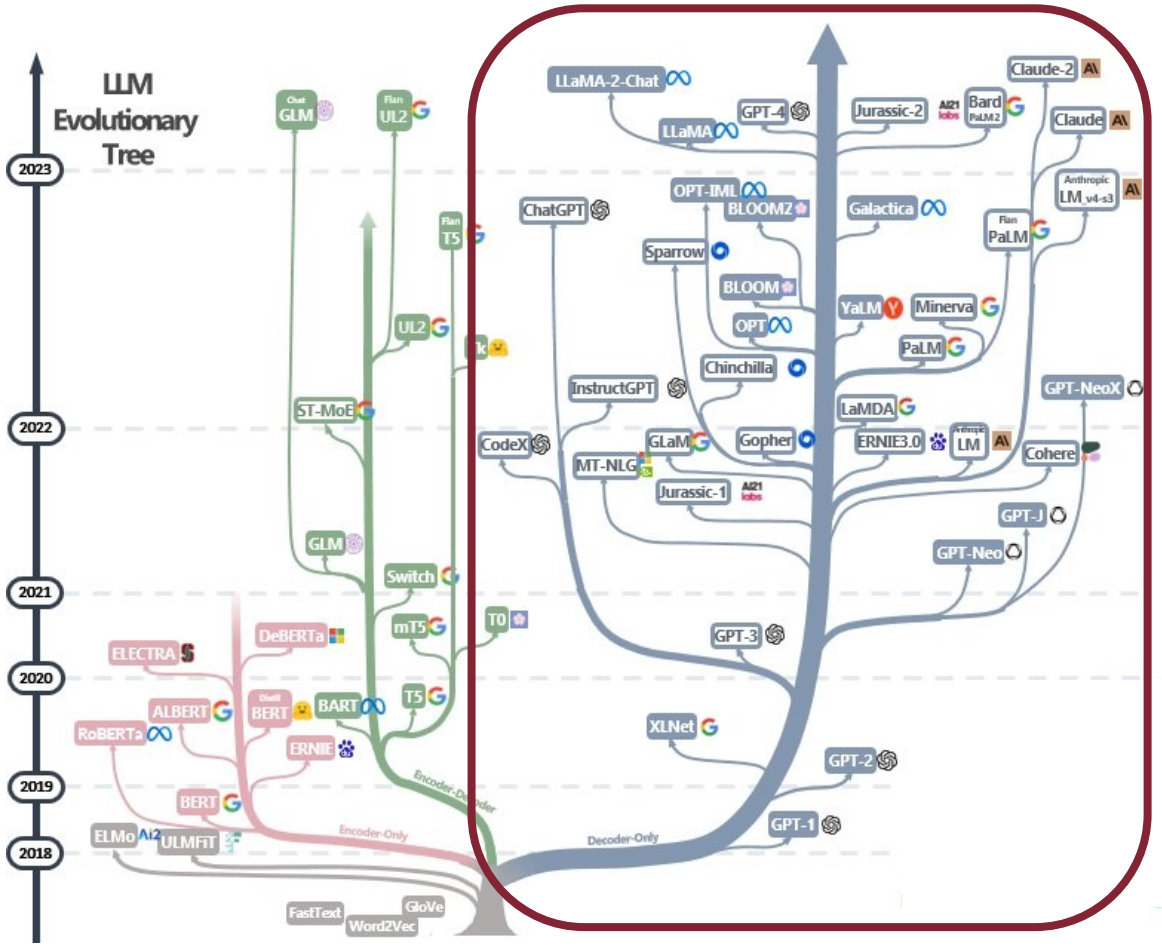


LLM - Architecture and Evolution

Decoder-Only Architecture

The transformer decoder architecture is used for tasks like **language generation**. The decoder takes in a fixed-size vector representation of the context and uses it to generate a sequence of words, with each word being conditioned on the previously generated words





Large Language Models - Pre-Training

Pre-Training or Training?

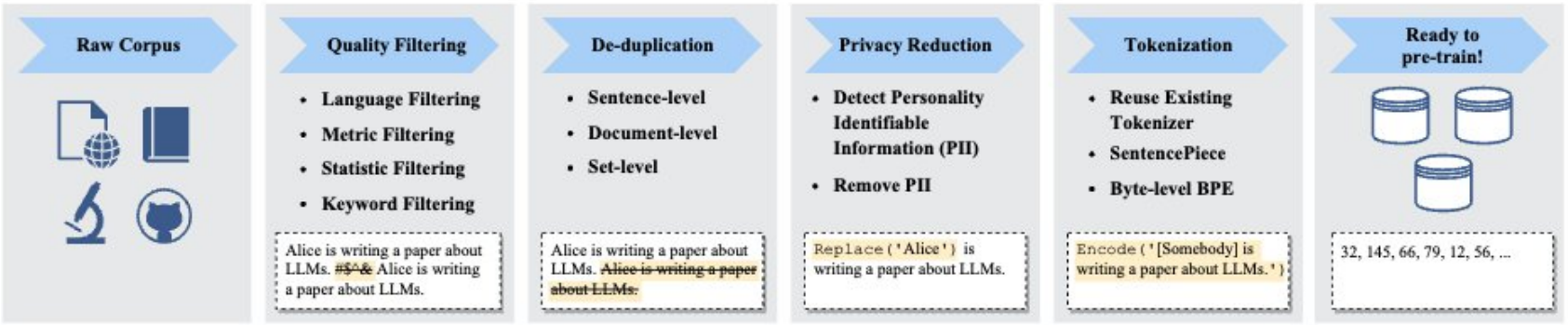
Pre-training is a foundational step in the LLM training process, where the model gains a general **understanding of language** by exposure to vast amounts of text data, in an unsupervised manner. It is called pre-training (instead of training) because after this stage the model is generally **not capable of solving specific problems**. Most pre-training common task are:

Masked Language Modeling (MLM): “The quick brown fox jumps over the _ _ _ _ (lazy) dog.”

Next Sentence Prediction (NSP): “The scientist made a groundbreaking discovery. _ _ _ (She was awarded the Nobel Prize).”

Contrastive Learning: “Hope is the thing with feathers — / That perches in the soul — / And sings the tune without the words — / And never stops at all — (Similar)”

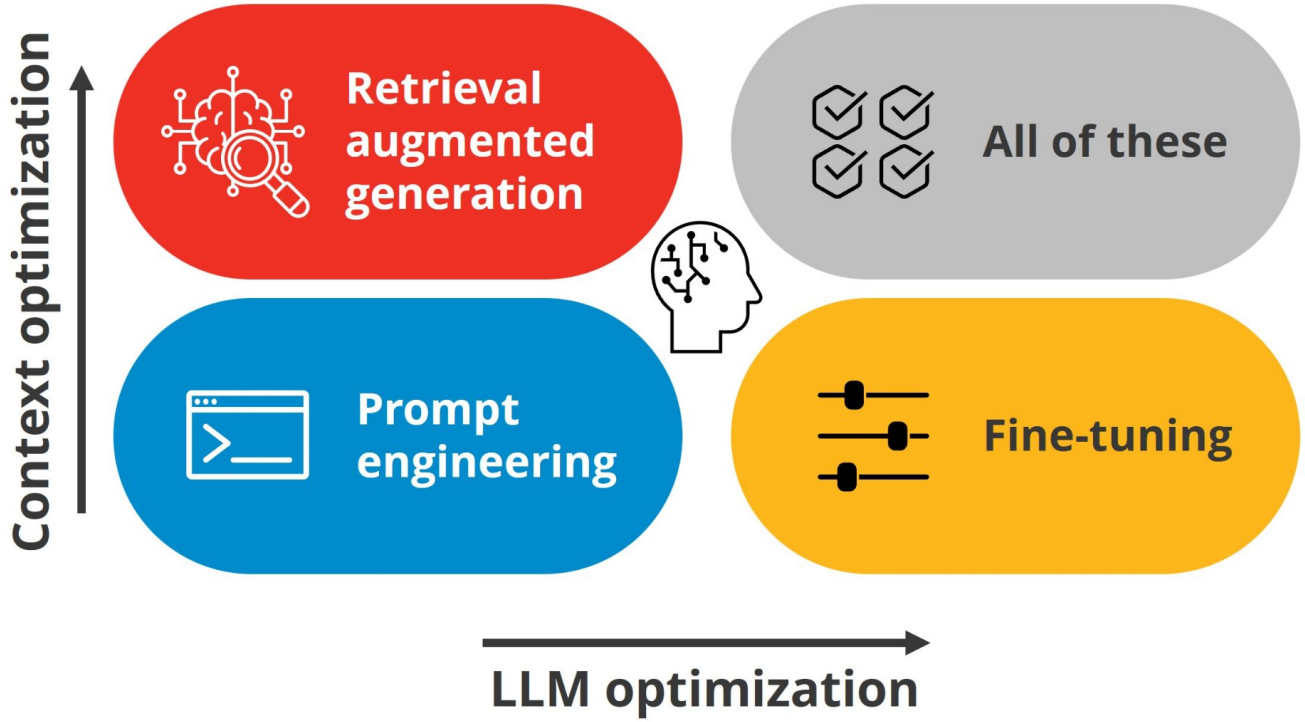
Permutation Language Modeling (PLM): “the sky blue is so” -> “so blue is the sky”



Pre-Training Pipeline[1]

[1] Zhao, Wayne Xin, et al. "A survey of large language models." arXiv preprint arXiv:2303.18223 (2023)

Specializing Large Language Models



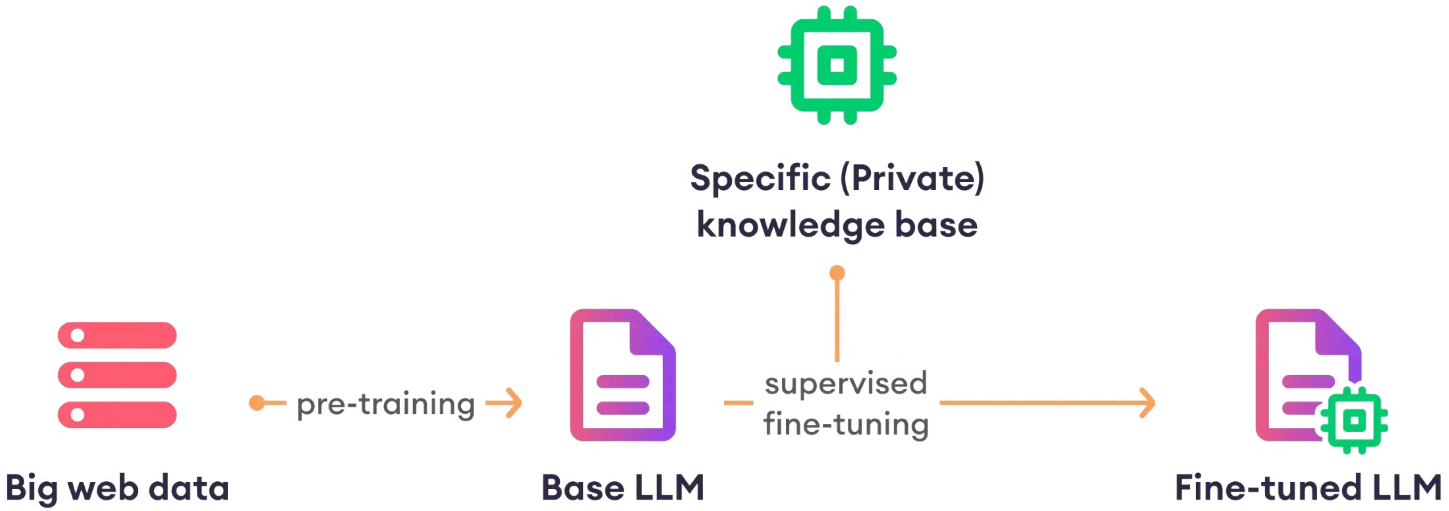
Optimizing a Language Model for a specific task

Source: https://fractal.ai/blog/governing-llms-through-enhancing-techniques/

Fine-Tuning

After pre-training, LLMs can acquire the general abilities for solving various tasks. However, an increasing number of studies have shown that LLM's abilities can be further adapted according to specific goals with fine tuning.

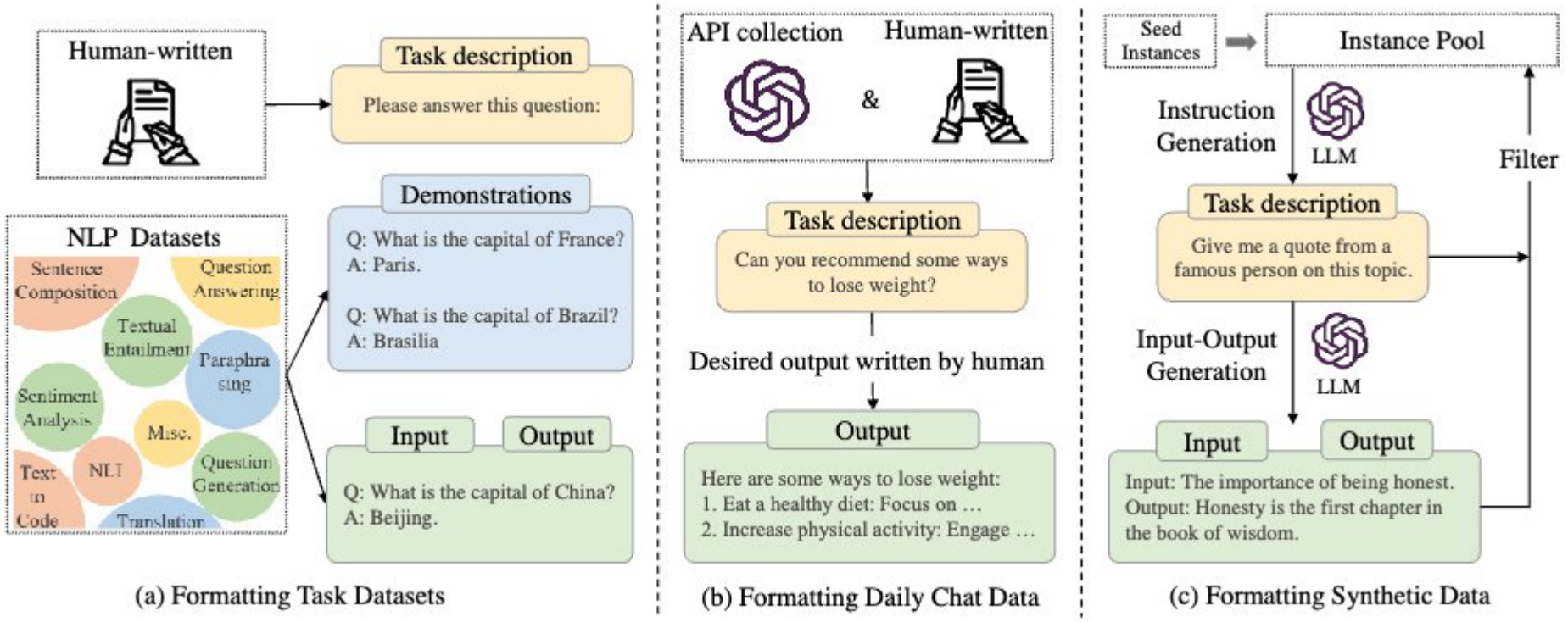
Several optimization strategies can make the fine-tuning process **parameter efficient** and/or **memory efficient**



Source:https://www.superannotate.com/blog/llm-fine-tuning

Fine Tuning - Instruction

Instruction tuning is the approach to fine-tuning pre-trained LLMs on a collection of formatted instances in the form of instructions in natural language. Even with a small dataset it can be very efficient



Instruction Fine tuning[1]

[1] Zhao, Wayne Xin, et al. "A survey of large language models." arXiv preprint arXiv:2303.18223 (2023)

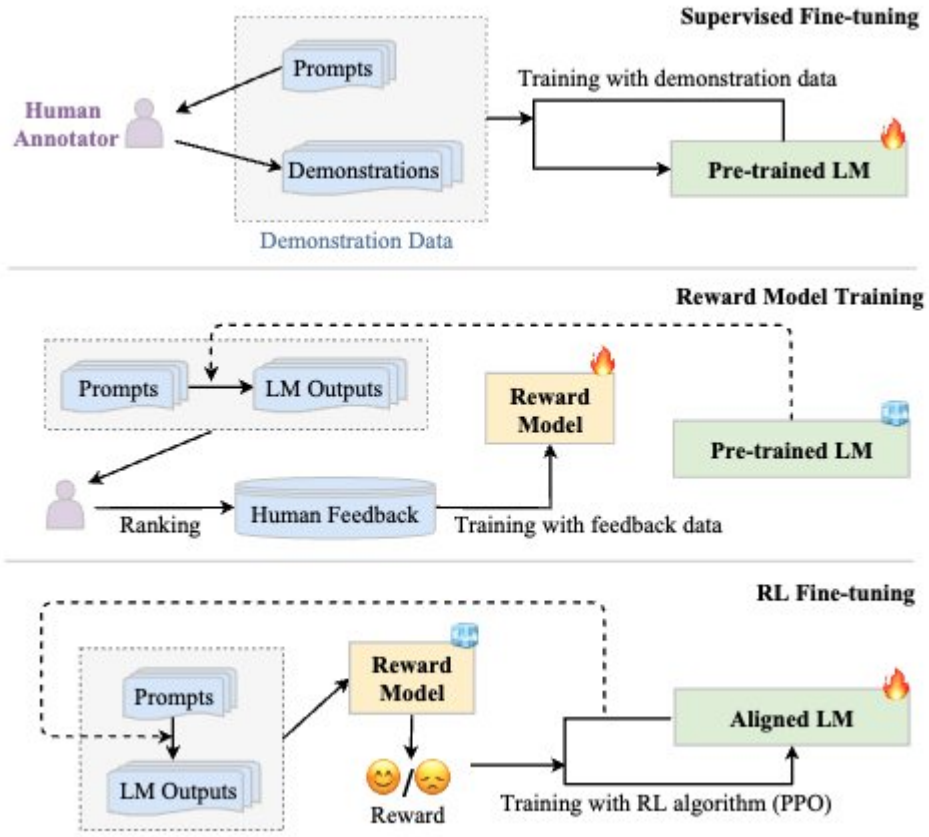
Fine Tuning - Alignment Tuning/RLHF

Alignment Tuning is used to avert these unexpected behaviors (*e.g.*, fabricating false information, pursuing inaccurate objectives, and producing harmful, misleading, and biased expression)

Reinforcement Learning from Human Feedback (RLHF)

It is used to align LLMs with human values. It is a fine-tuned LLMs with the collected human feedback data, which is useful to improve the alignment criteria (*e.g.*, helpfulness, honesty, and harmlessness). employing reinforcement learning (RL) algorithms to adapt LLMs to human feedback by learning a reward model.

- ChatGPT is an example



RLHF pipeline[1]

[1] *Zhao, Wayne Xin, et al. "A survey of large language models." arXiv preprint arXiv:2303.18223 (2023)*

SAPIENZA UNIVERSITÀ DI ROMA

Fine Tuning limitations

Specialization is expensive

Fine-tuning is the **most effective** technique for specializing an LLM. However, it is **not always viable**. The major drawbacks of fine-tuning of LLM are **costs** in terms of:

- human resources (for both supervised and unsupervised training)
- time
- power
- CO2 emissions

Model		GPT-3	BLOOM	LLaMA	LLaMA-2	T5	PaLM
Developer		OpenAI	BigScience	Meta		Google	
Model Size (# parameters)		175B	175B	7B, 13B, 33B, 65B	7B, 13B, 34B, 70B	11B	540B
Training Data (# tokens)		300B	350B	1.4T	2T	34B	795B
Training Compute (FLOPs)		3.2E+23	3.7E+23	9.9E+23	1.5E+24	2.2E+21	2.6E+24
Processor	Manufacturer	Nvidia	Nvidia	Nvidia	Nvidia	Google	Google
	Type	GPU	GPU	GPU	GPU	TPU	TPU
	Model	V100	A100	A100	A100	TPU v3	TPU v4
Processor Hours		3,552,000	1,082,990	1,770,394	3,311,616	245,760	8,404,992
Grid Carbon Intensity (kgCO2e/KWh)		0.429	0.057	0.385	0.423	0.545	0.079
Data Center Efficiency (PUE)		1.1	1.2	1.1	1.1	1.12	1.08
Energy Consumption (MWh)		1,287	520	779	1,400	86	3,436
Carbon Emissions (tCO2e)		552	30	300	593	47	271

$$KWh = Hours\ to\ train \times Number\ of\ Processors \times Average\ Power\ per\ Processor \times PUE \div 1000$$

$$tCO2e = KWh \times kg\ CO2e\ per\ KWh \div 1000$$

Source: https://lajavaness.medium.com/llm-large-language-model-cost-analysis-d5022bb43e9e

Fine Tuning - Parameter Efficient

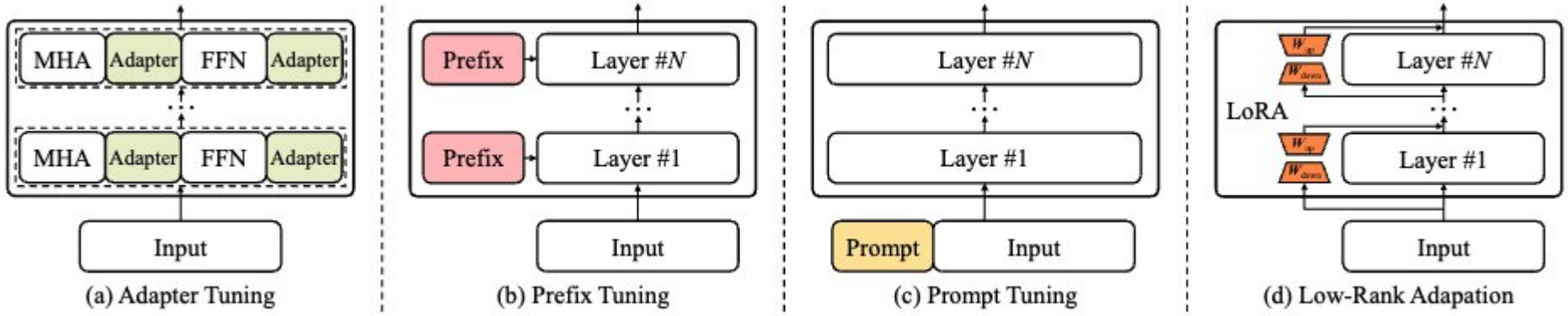
Parameter-efficient fine-tuning has been an important topic that aims to reduce the number of trainable parameters while retaining a good performance as possible. Several techniques can be used:

Adapter Tuning: incorporates **small neural network modules** (called adapter) into the Transformer models. Only adapter parameters are updated during fine-tuning

Prefix Tuning: prepends a sequence of prefixes, which are a set of **trainable continuous vectors**, to each Transformer layer in language models. Only prefix parameters are updated during fine-tuning

Prompt Tuning: incorporates **trainable prompt vectors** at the input layer

Low-Rank Adaptation (LoRA): imposes the **low-rank constraint** for approximating the update matrix at each dense layer, to reduce the trainable parameters for adapting to downstream tasks



Parameter efficient fine tuning[1]

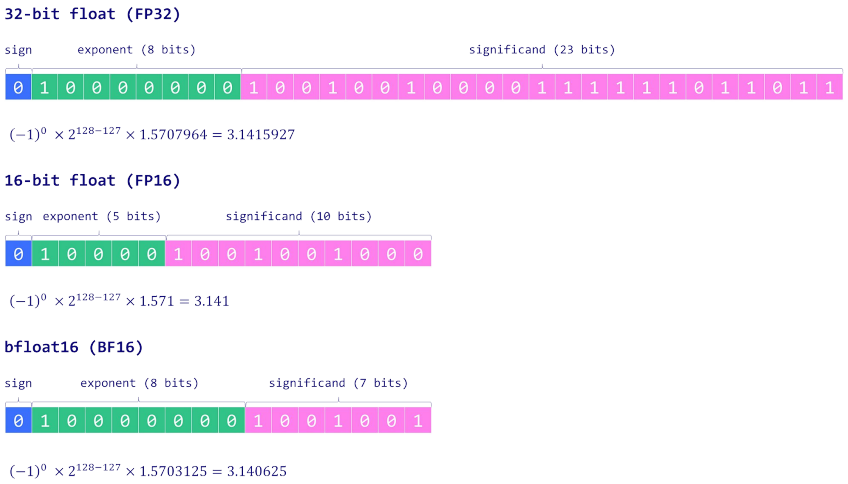
[1] Zhao, Wayne Xin, et al. "A survey of large language models." arXiv preprint arXiv:2303.18223 (2023)

Fine Tuning - Memory Efficient

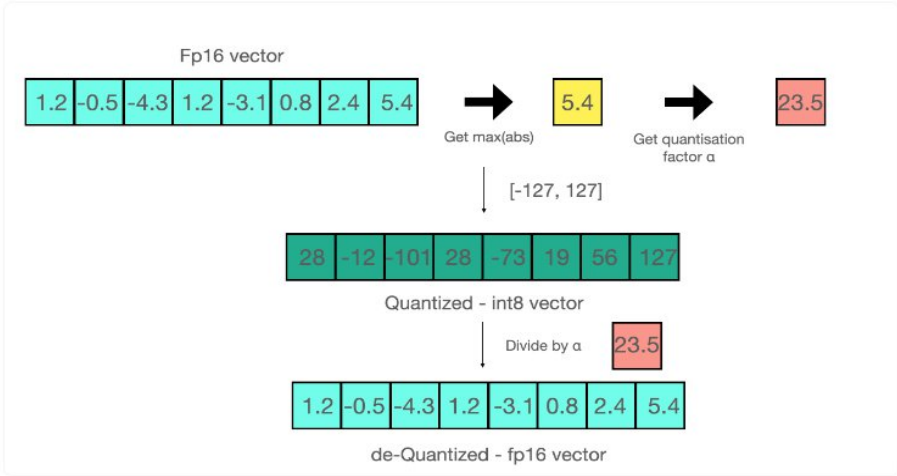
The most used techniques to reduce a model's memory footprint are either reducing floating points weight resolution from the full 32-bit down to 16-bit or quantization

Quantization

It maps weights from floating-point numbers to integers, of different resolutions, from 8-bit down to 2-bit



Source:
https://towardsdatascience.com/introduction-to-weight-quantization-2494701b9c0c



Source: https://levelup.gitconnected.com/cant-afford-the-expensive-gpu-for-your-ai-development-try-4-bit-quantization-b26108229b75

Prompt Engineering

Prompt engineering is the process of structuring an instruction that can be interpreted and understood by a generative AI model[1]. It enables:

Zero-Shot

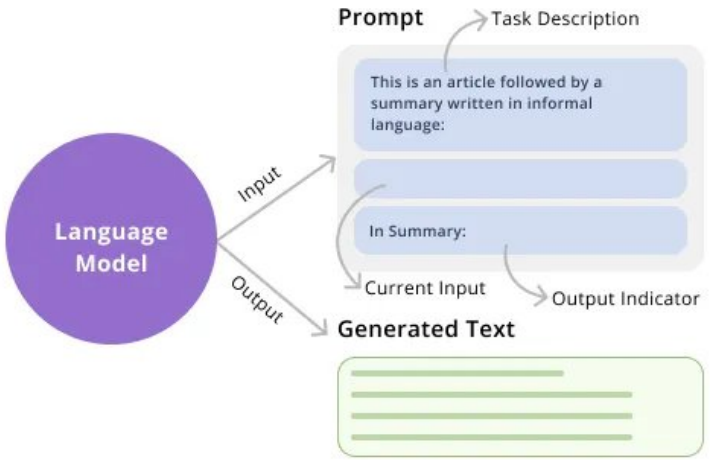
The prompt used to interact with the model **won't contain examples** or demonstrations

In-Context Learning or Few-Shot Learning

Demonstrations of the task are provided to the model as part of the prompt

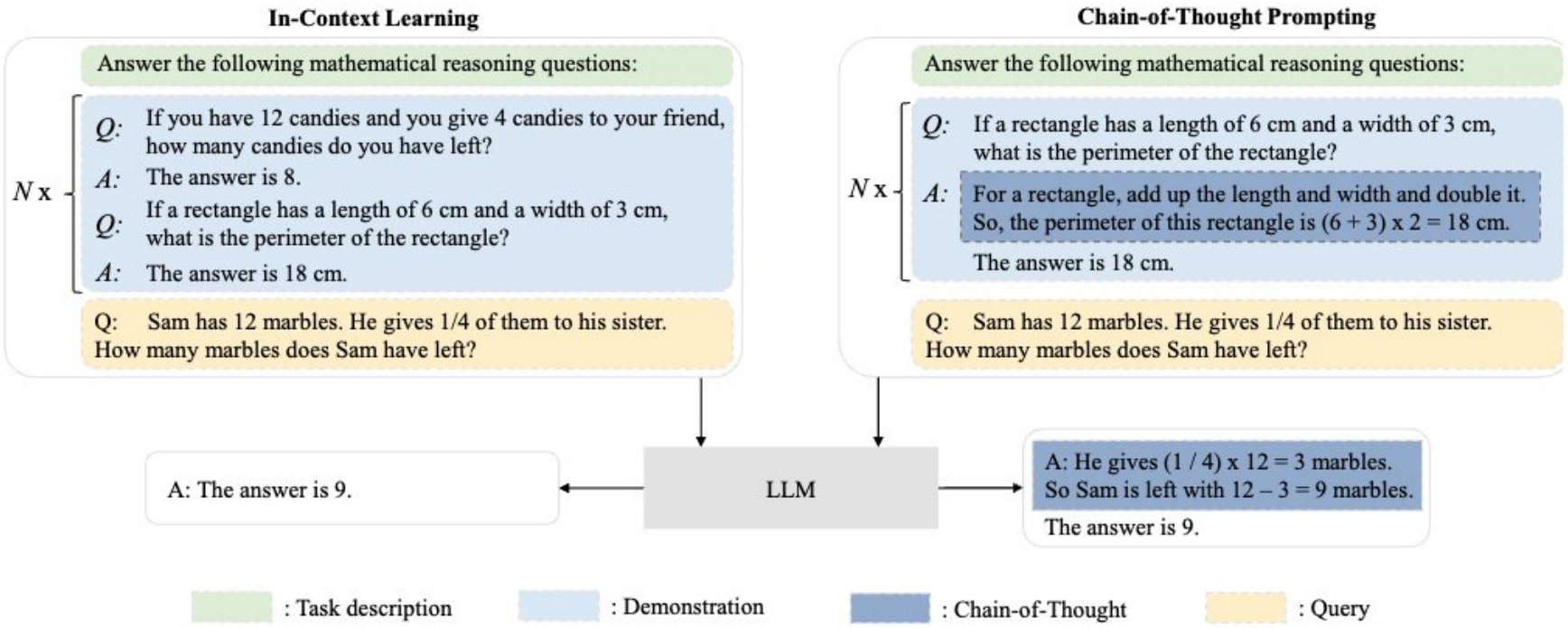
Chain of thought/reasoning

Incorporates intermediate **reasoning steps**, which serve as the bridge between inputs and outputs



Source: https://botpenguin.com/glossary/prompt-engineering

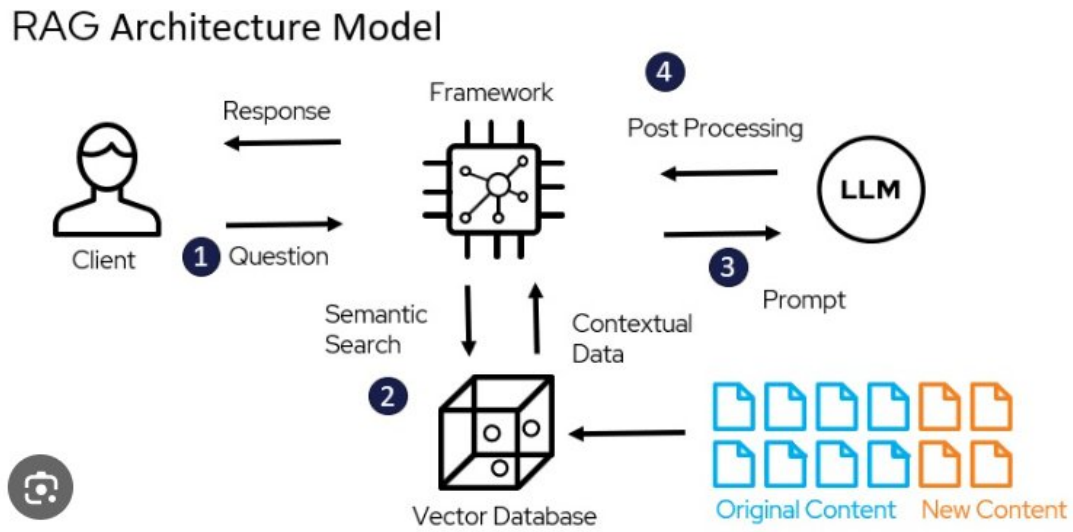
Prompt Engineering at Work



Source: [Zhao, Wayne Xin, et al. "A survey of large language models." arXiv preprint arXiv:2303.18223 (2023)]

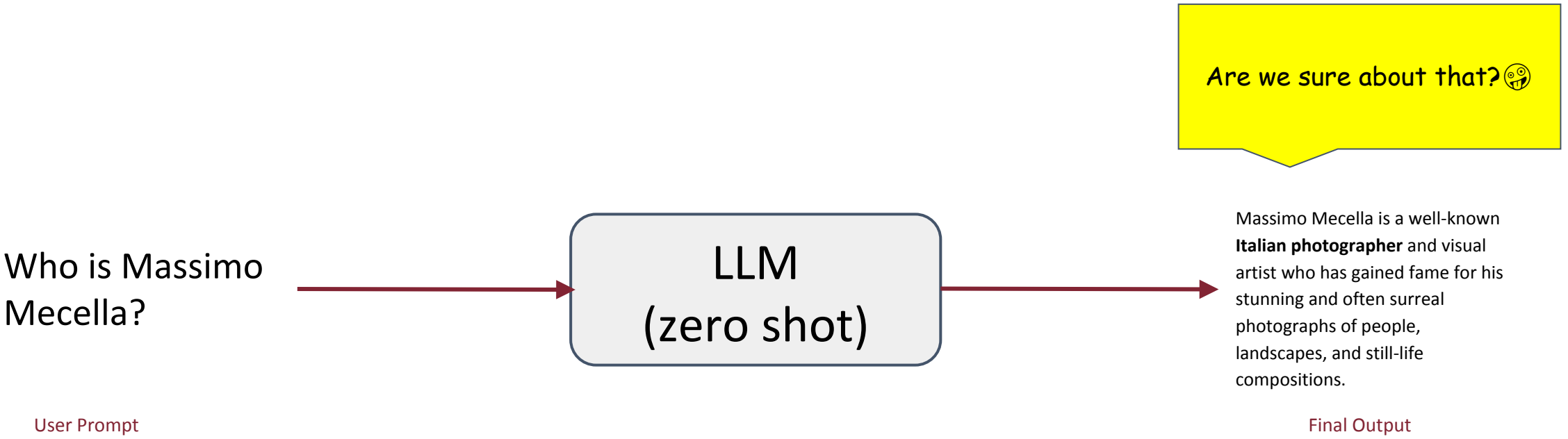
Retrieval Augmented Generation (RAG)

RAG takes an input and retrieves a set of r**elevant/supporting documents** given a source. The documents are **concatenated as context** with the original input prompt and fed to the text generator which produces the final output.



Source:
https://medium.com/@bijit211987/advanced-rag-for-llms-slms-5bcc6fbba411

RAG at Work



Tested with [meta-llama/Meta-Llama-3-8B-Instruct]

RAG at Work

Who is Massimo Mecella?

User Prompt

Retrieval from the internet

LLM + RAG

You are an assistant for question-answering tasks. Use the following pieces of retrieved context to answer the question. If you don't know the answer, just say that you don't know. Use three sentences maximum and keep the answer concise.
Question: Who is massimo mecella?
Context: Massimo Mecella | Department of Computer, Control and Management Engineering [...]

Rag generated prompt

Much better 🙏

Massimo Mecella is a **Professor of computer science** at the Sapienza University of Rome, Italy, and leader of the Processes, Services and Software Management group.

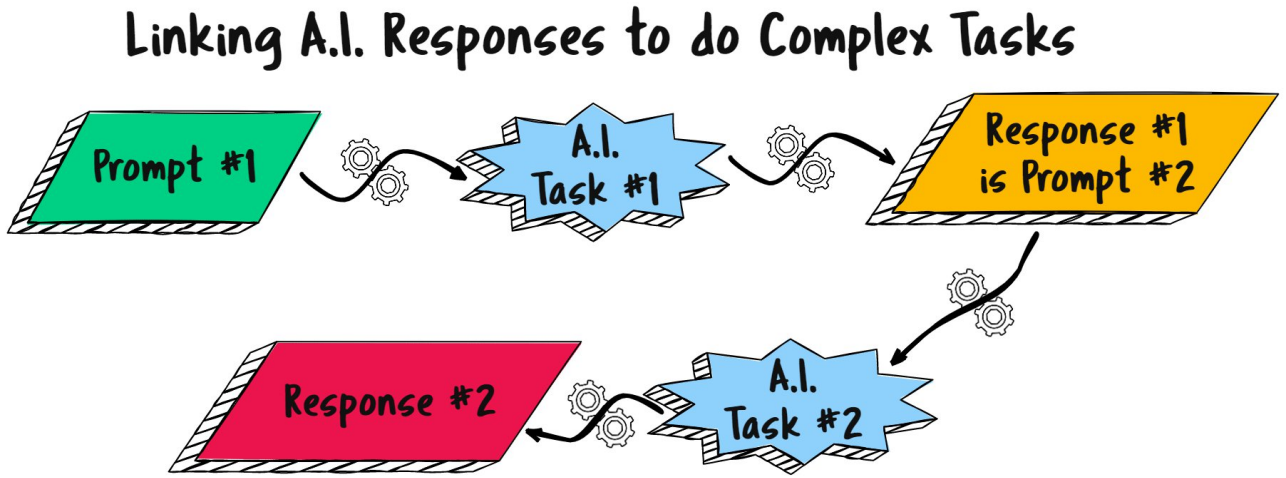
Final Output

Tested with [meta-llama/Meta-Llama-3-8B-Instruct] and [langchain v0.2]

SAPIENZA UNIVERSITÀ DI ROMA

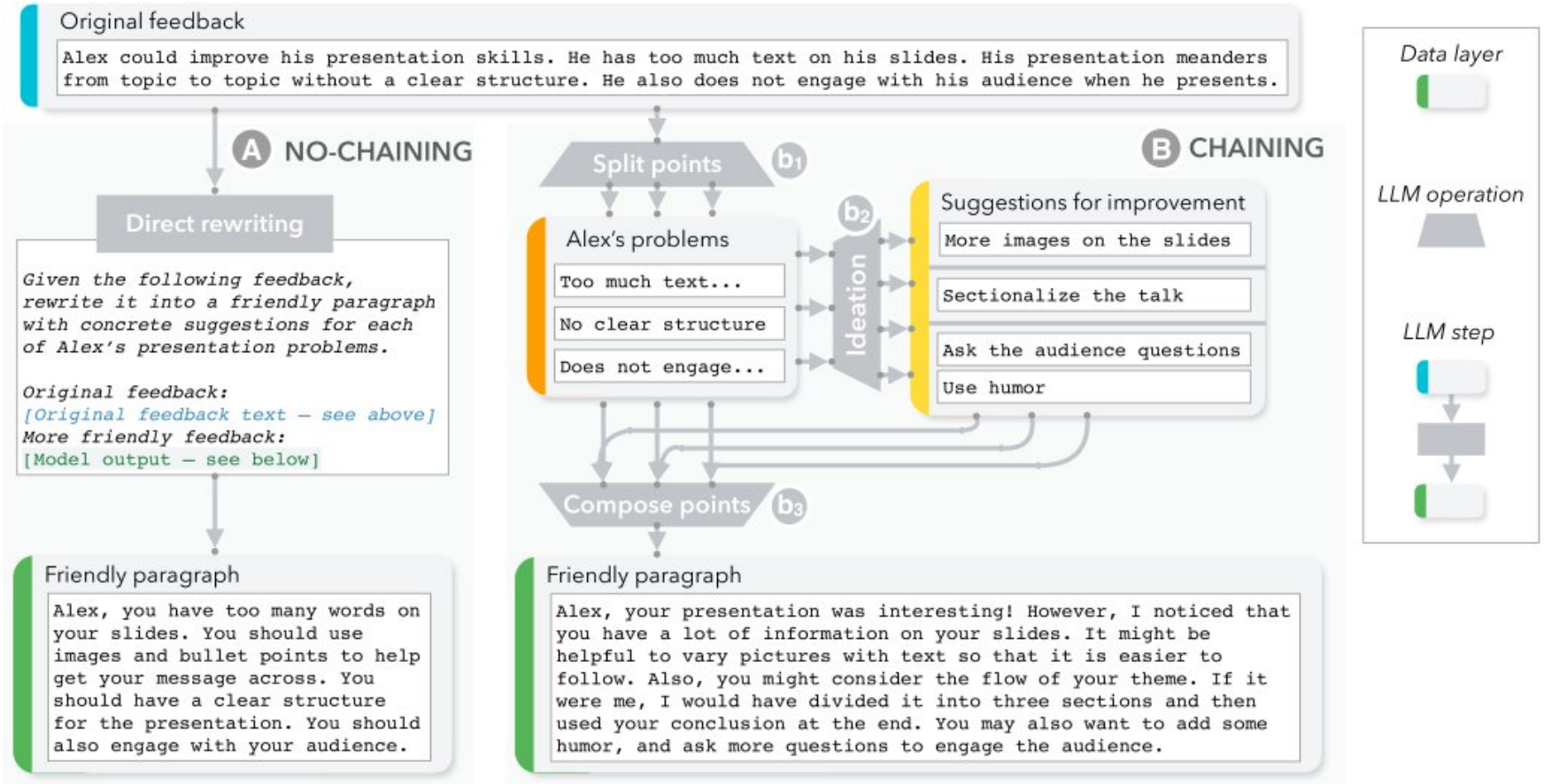
Prompt Chaining

To improve the reliability and performance of LLMs, one of the important prompt engineering techniques is **to break tasks into its subtasks**. Once those subtasks have been identified, the LLM is prompted with a subtask and then its response is used as input to another prompt.

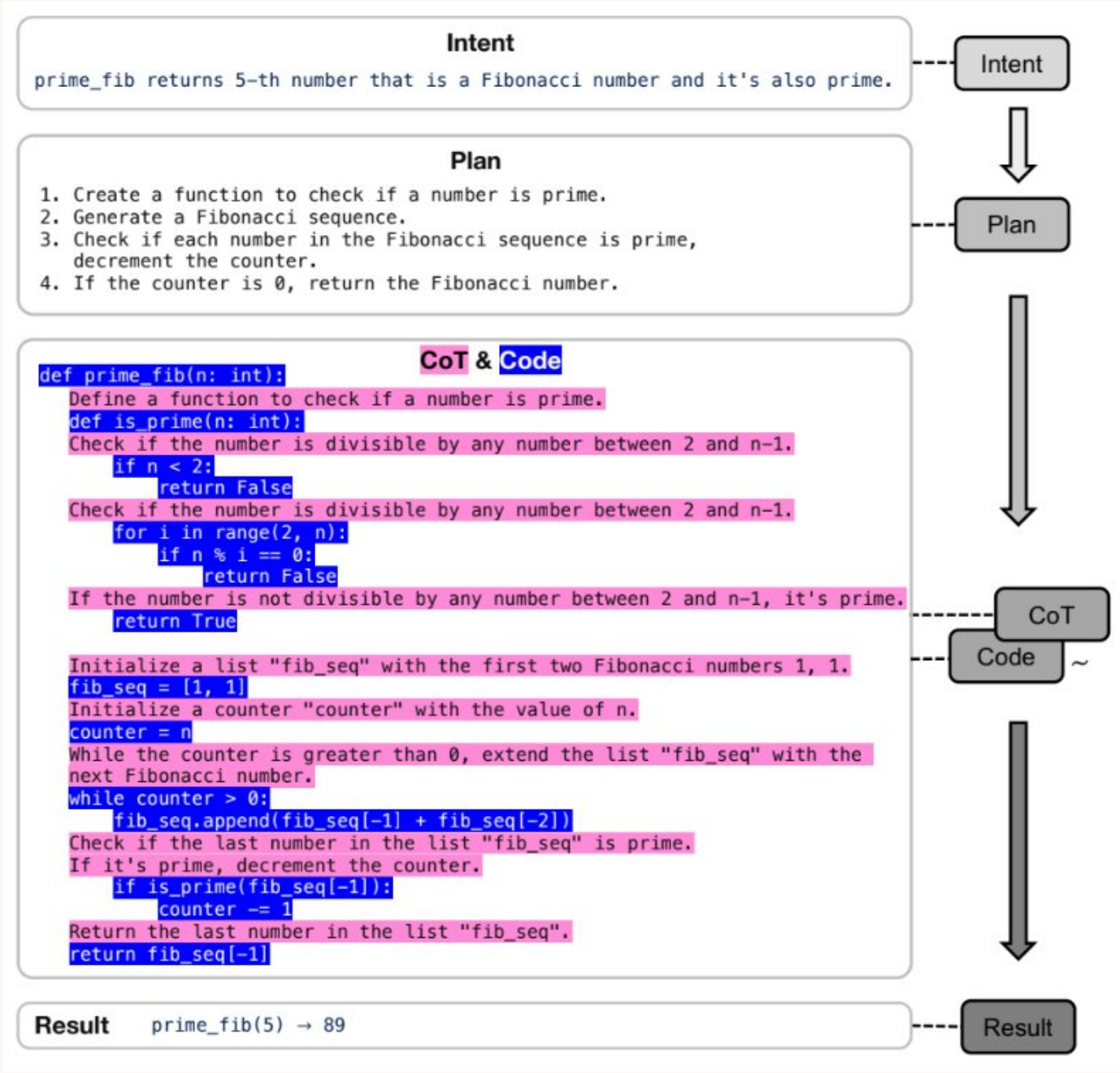


Source:
https://aigenealogyinsights.com/2023/06/13/the-power-of-a-i-genealogical-prompt-chaining/

Prompt Chaining at Work

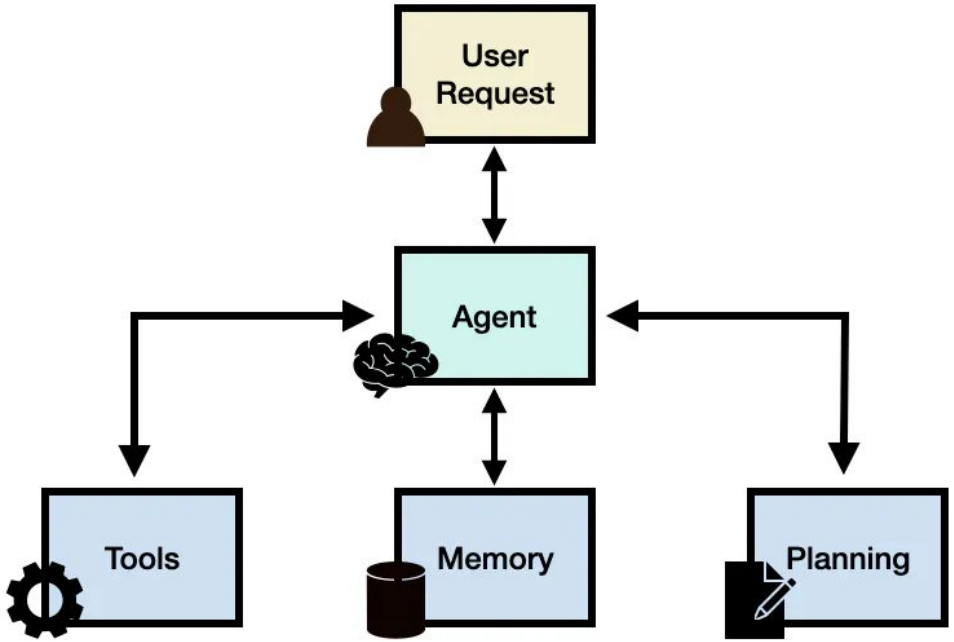


Source:
https://cobusgreyling.medium.com/chaining-large-language-model-prompts-81091daccaad


Intent
prime_fib returns 5-th number that is a Fibonacci number and it's also prime.
Plan
1. Create a function to check if a number is prime.
2. Generate a Fibonacci sequence.
3. Check if each number in the Fibonacci sequence is prime, decrement the counter.
4. If the counter is 0, return the Fibonacci number.
CoT & Code
def prime_fib(n: int):
Define a function to check if a number is prime.
def is_prime(n: int):
Check if the number is divisible by any number between 2 and n-1.
if n < 2:
return False
Check if the number is divisible by any number between 2 and n-1.
for i in range(2, n):
if n % i == 0:
return False
If the number is not divisible by any number between 2 and n-1, it's prime.
return True
Initialize a list "fib_seq" with the first two Fibonacci numbers 1, 1.
fib_seq = [1, 1]
Initialize a counter "counter" with the value of n.
counter = n
While the counter is greater than 0, extend the list "fib_seq" with the next Fibonacci number.
while counter > 0:
fib_seq.append(fib_seq[-1] + fib_seq[-2])
Check if the last number in the list "fib_seq" is prime.
If it's prime, decrement the counter.
if is_prime(fib_seq[-1]):
counter -= 1
Return the last number in the list "fib_seq".
return fib_seq[-1]
Result
prime_fib(5) → 89
Intent
Plan
CoT
Code
Result

LLM Agent

LLM agents involve LLM applications that can execute **complex tasks** using an architecture that combines LLMs with key modules like **planning and memory**. When building LLM agents, an LLM serves as the **main controller** or "brain" that controls a flow of operations needed to complete a task or user request



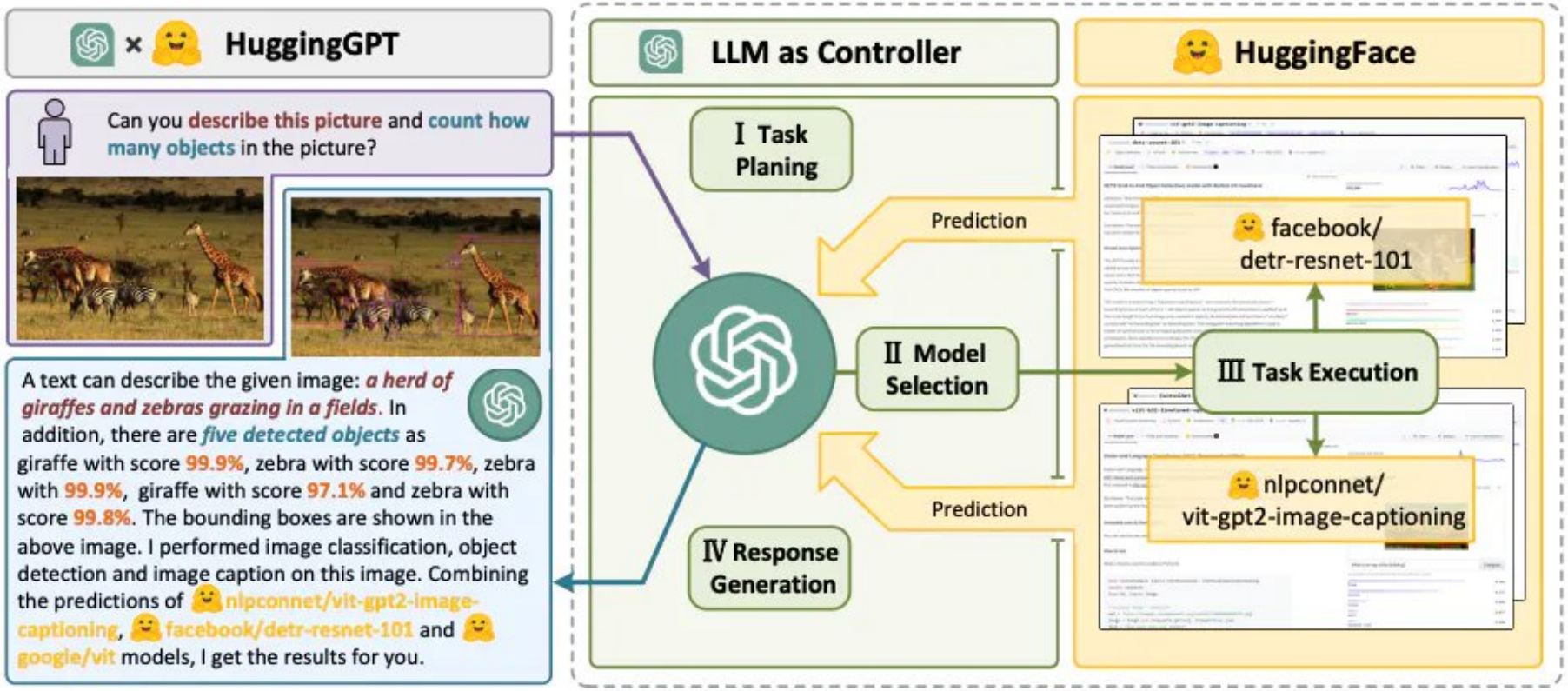
Source: https://www.promptingguide.ai/research/llm-agents

```
agent.run("Caption the following image", image=image)
```

Input	Output
	A beaver is swimming in the water

Source: https://huggingface.co/docs/transformers/transformers_agents

LLM Agent at Work



Source: https://www.promptingguide.ai/research/llm-agents.en

LLM Agent at Work II

Example

User:

① Find the sum of the reciprocals of the roots of $x^2-13x+4=0$.

Assistant:

② To find the sum of the reciprocals of the roots of the quadratic equation $x^2-13x+4=0$ [...]

③
```
<execute>
import sympy
x = sympy.Symbol('x')
roots = sympy.solve(x**2 - 13*x + 4)
print(1/roots[0] + 1/roots[1])
</execute>
```

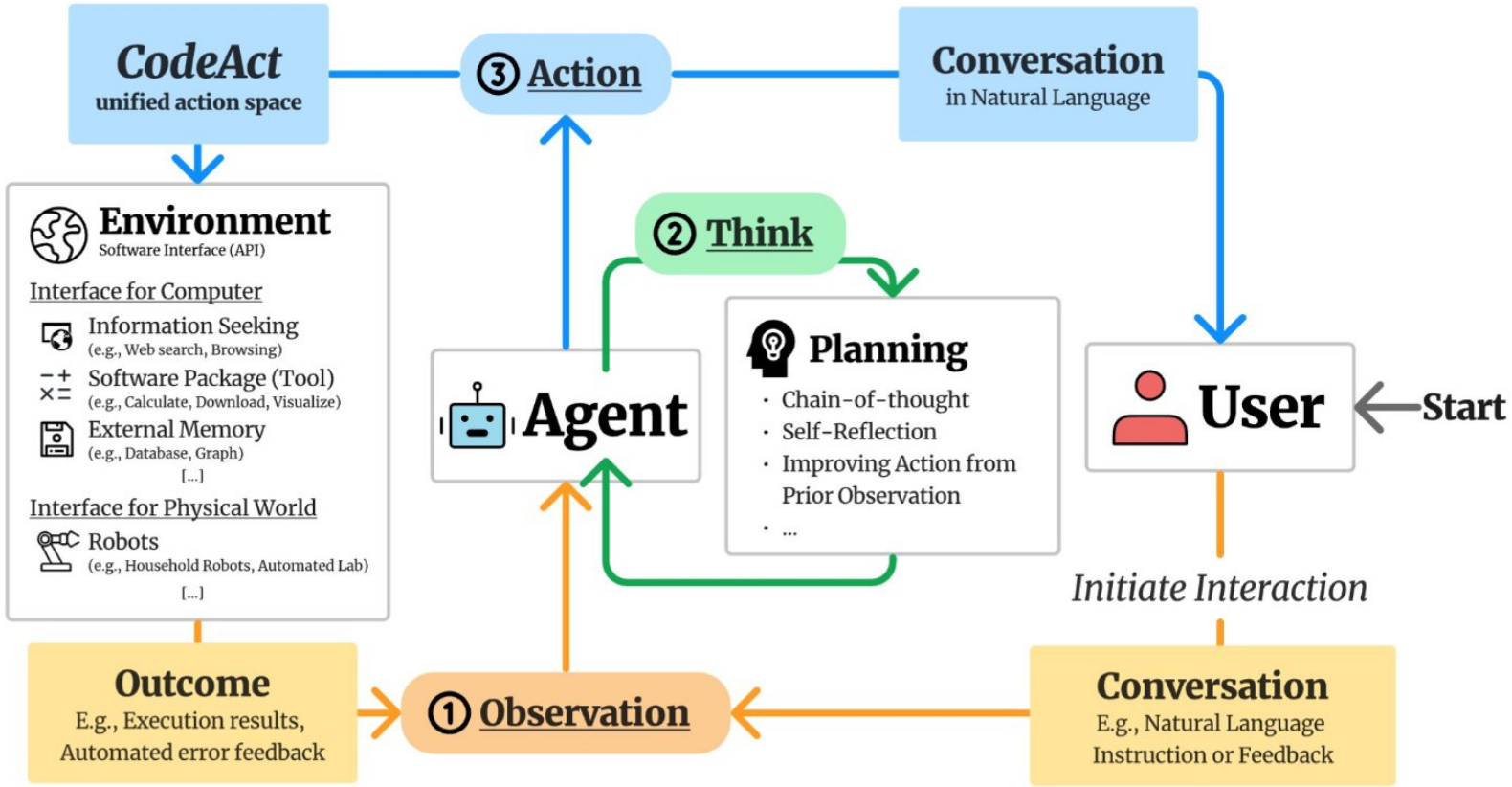
Env:

① 1/(3*sqrt(17)/2 + 13/2) + 1/(13/2 - 3*sqrt(17)/2)

Assistant:

② The roots obtained are in symbolic form, which makes it difficult to evaluate the sum directly. Let's use the quadratic formula to explicitly find the numerical values of the roots first.

[...]



AI Agents for Code Development

Github Copilot

GitHub Copilot is an AI-powered pair programmer that acts as a coding assistant to help developers write code faster and more efficiently. It provides context-aware code suggestions as you type, can write code from natural language prompts, and assists with various tasks like debugging, creating unit tests, and explaining code

Windsurf

Windsurf AI is an AI-powered integrated development environment (IDE) and coding assistant that enhances developer productivity through features like code generation, intelligent suggestions, and multi-file editing

Cursor

Cursor AI is a code editor that integrates AI directly into a familiar VS Code-like interface to help developers write code faster using natural language